scienceStack

Rigorous Agentic Engineering & Theoretical Computing

A Deep Integration and Operational Manual for Google Antigravity

---

Plugin Specification & Operational Guide

Version 0.15.29 — October 2026

Core Architecture:
35 Top-Level Skills
30 Bundled Core Principles
3 Dedicated Subagents
1 Unified Runtime Rule
1 Pre-Tool Safety Hook

Theoretical Domains:
PDEs, CFD, Finite Elements
Stiff ODE Kinetics, MD
Quantum Systems, Linear Algebra

Adapted from the foundational `pstack` architecture by Lauren Tan (poteto) at Cursor.
Expanded for rigorous theoretical fidelity, dimensional analysis, and numerical verification in Google Antigravity.

Contents

Executive Summary iii

1 System Overview and Core Philosophy 1
- 1.1 The Crisis of Slop in Scientific Computing . . . 1
- 1.2 The scienceStack Philosophy . . . 1
  - 1.2.1 Fearless Parallelism . . . 1
- 1.3 Architectural Heritage: From pstack to scienceStack . . . 2

2 Installation, Setup, and Model Routing 3
- 2.1 Installing the Plugin . . . 3
  - 2.1.1 Global Installation . . . 3
  - 2.1.2 Verification of Installed Components . . . 3
- 2.2 Model Configuration and Reasoning Budgets . . . 3
  - 2.2.1 The Role-Based Model Architecture . . . 3
  - 2.2.2 Interactive Setup with /setup-sciencestack . . . 4

3 The Specialized Subagent Ecosystem 5
- 3.1 Overview of Agent Roles . . . 5
- 3.2 science-master: The Scientific Implementation Delegate . . . 5
- 3.3 theory-reviewer: The Mathematical Auditor . . . 6
  - 3.3.1 The 6-Point Theoretical Audit Checklist . . . 6
- 3.4 comment-master: Eliminating Commentary Rot . . . 7

4 The Six Core Scientific Principles 8
- 4.1 Principle 1: Equation Provenance and Formal Documentation . . . 8
  - 4.1.1 Code Comparison: Bad vs. Compliant . . . 8
- 4.2 Principle 2: Dimensional Discipline & Scaling . . . 9
- 4.3 Principle 3: Preservation of Physical Conservation Laws . . . 9
- 4.4 Principle 4: Numerical Stability First . . . 10
- 4.5 Principle 5: Verification Against Analytical Limits and Golden Benchmarks . . . 10
- 4.6 Principle 6: Physics-Modular Architecture & Decoupled Operators . . . 11
  - 4.6.1 Architectural Comparison: Monolithic vs. Modular . . . 11

5 Scientific Skills and Playbooks 13
- 5.1 The Master Controller: /science-mode . . . 13
- 5.2 Dedicated Scientific Skills . . . 13
  - 5.2.1 /scientific-equation-verifier . . . 13
  - 5.2.2 /numerical-stability-audit . . . 13
  - 5.2.3 /golden-dataset-benchmark . . . 14
- 5.3 Specialized Scientific Playbooks . . . 14
  - 5.3.1 Playbook: equation-implementation . . . 14

5.3.2 Playbook: numerical-verification . . . 15

6 Astronomy and Celestial Mechanics Solvers 16
6.1 Overview of the Astronomy Suite . . . 16
6.2 Kepler Orbit Propagation and Vis-Viva Invariants . . . 16
6.2.1 Mathematical Formulation . . . 16
6.2.2 Cubic Halley Solver . . . 17
6.2.3 Invariant Verification . . . 17
6.3 Symplectic Gravitational N-Body Dynamics . . . 17
6.3.1 The Hamiltonian and Symplectic Structure . . . 17
6.3.2 Drift-Kick-Drift Leapfrog . . . 17
6.4 Stellar Structure: Lane-Emden Polytropes . . . 18
6.4.1 Governing Equation and Singularity Removal . . . 18
6.4.2 Analytical and Tabulated Benchmarks . . . 18

7 Automated Tooling and Antigravity Lifecycle Hooks 19
7.1 The AST Numerical Stability Auditor . . . 19
7.2 Symbolic Method of Manufactured Solutions (MMS) Generator . . . 19
7.3 Symbolic Jacobian Synthesizer . . . 20
7.4 Native Antigravity Lifecycle Hook: PreToolUse Safety Gate . . . 20

8 End-to-End Practical Tutorials 21
8.1 Tutorial 1: Implementing a Stiff Kinetic System (Robertson) . . . 21
8.1.1 Problem Statement . . . 21
8.1.2 Executing with /science-mode . . . 21
8.1.3 What Happens Internally . . . 21
8.2 Tutorial 2: Conducting an Overnight Autonomous Run . . . 22
8.2.1 The Overnight Contract Prompt . . . 22
8.2.2 Safeguards During Autonomous Execution . . . 22
8.3 Tutorial 3: Verifying Symplectic N-Body Orbit Invariants . . . 22
8.3.1 Problem Statement . . . 22
8.3.2 Executing with /science-mode . . . 22
8.3.3 Verification and Invariant Output . . . 23

9 Antigravity Technical Adapter Reference 24
9.1 Runtime Tool Mapping . . . 24
9.2 Antigravity Brain Artifacts . . . 24

10 Complete Skill and Playbook Catalog 25
10.1 Primary Scientific and Orchestration Skills . . . 25
10.2 Scientific and Engineering Principles Catalog . . . 25
10.3 General Engineering and Review Skills . . . 27

Recommended Literature & References 28

Executive Summary

In traditional software engineering, artificial intelligence coding assistants often generate superficial code that satisfies local syntax tests but fails under boundary conditions. In **computational science and engineering**, the cost of generated "slop" is catastrophic. A flipped negative sign in a Laplacian diffusion operator turns a dissipative physical process into an exponentially explosive numerical instability. A subtle catastrophic cancellation in floating-point subtraction silently destroys the condition number of a linear solve. An unverified normalization constant renders quantum wavefunctions non-unitary.

scienceStack is an Antigravity-native agentic framework engineered specifically to eliminate ungrounded code in scientific and mathematical computing. Originating from the battle-tested `pstack` architecture created by Lauren Tan at Cursor, `scienceStack` completely re-architects the agent pairing paradigm for Google Antigravity:

1. **Theoretical Provenance**: Every formula implemented by an agent must be explicitly derived in LaTeX docstrings, cite canonical literature equations, and define coordinate metrics and sign conventions.
2. **Dimensional Discipline**: Every equation is verified for physical dimension matching ($[\text{LHS}] = [\text{RHS}]$), and transcendental arguments ($\exp, \ln, \sin$) are enforced to be dimensionless.
3. **Conservation Law Guarantees**: Numerical schemes are required to track and conserve physical invariants (energy, mass, momentum, probability norm) within specified tolerance thresholds.
4. **Numerical Stability First**: The framework strictly bans naive mathematical implementations (e.g., explicit matrix inversion $\mathbf{A}^{-1}\mathbf{b}$, unscaled floating-point differences) in favor of conditioned algorithms (`solve(A, b)`, `logsumexp`, `expm1`).
5. **Analytical Limits & Golden Datasets**: Every scientific kernel must be validated against closed-form exact solutions, the Method of Manufactured Solutions (MMS), or canonical industry benchmark datasets (e.g., Sod shock tube, Taylor-Green vortex, Robertson stiff ODEs).
6. **Physics-Modular Architecture**: Physical laws, equations of state, forces, and rate equations must be isolated into dedicated, driver-agnostic modules (`physics/`) so other simulation codes and external solvers can invoke them directly without coupling to numerical time steppers or spatial grids.

With 35 top-level skills, 30 bundled scientific and software engineering principles, three specialized subagent personas (`science-master`, `theory-reviewer`, and `comment-master`), and native Antigravity asynchronous orchestration, `scienceStack` enables fearless parallel development while upholding the mathematical rigor demanded by modern science.

Chapter 1

System Overview and Core Philosophy

1.1 The Crisis of Slop in Scientific Computing

Modern Large Language Models (LLMs) excel at generating idiomatic code snippets, web components, and routine test scripts. However, when applied to computational physics, numerical linear algebra, biomechanics, or quantum chemistry, unassisted LLMs exhibit severe failure modes:

- **Hallucinated Equations**: Formulating plausible-looking discrete approximations that lack theoretical backing or violate fundamental symmetries.
- **Catastrophic Floating-Point Cancellation**: Evaluating expressions like $f(x) = \sqrt{x+1} - \sqrt{x}$ or $(1 - \cos x)$ for small x without using Taylor series or conditioned algebraic identities, leading to severe loss of significance.
- **Inversion of Ill-Conditioned Matrices**: Emitting $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ via `numpy.linalg.inv(A) @ b` instead of direct LU/Cholesky/QR solvers, squaring the matrix condition number $\kappa(\mathbf{A})$.
- **Violation of Noetherian Invariants**: Implementing time-integration schemes (such as standard forward Euler) for conservative Hamiltonian systems, causing artificial energy blowup over time.
- **Narrative Noise in Codebases**: Polluting clean mathematical code with conversational comments, redundant stubs, and defensive workarounds that obscure algorithmic clarity.

1.2 The scienceStack Philosophy

`scienceStack` operates on a single immutable maxim:

"If you want to go fast, go deep first."

Throughput without theoretical accuracy is counterproductive. Generating ten thousand lines of unverified simulation code produces negative value if the underlying discretization violates conservation laws.

1.2.1 Fearless Parallelism

In traditional development, delegating work to multiple concurrent background agents leads to merge conflicts, architectural drift, and silent bugs. `scienceStack` makes parallelism safe by establishing strict entry contracts, formal verification suites, and autonomous review panels:

- **Isolated Workspaces**: Concurrent subagents execute inside isolated Git branches or worktrees (`Workspace: "branch"`).
- **Theoretical Gatekeepers**: No scientific kernel is merged into the primary branch until audited by the independent `theory-reviewer` subagent.
- **Objective Golden Benchmarks**: Work is considered complete only when empirical convergence rates $O(h^p)$ or relative L_2 error thresholds against reference datasets are demonstrated.

1.3 Architectural Heritage: From pstack to scienceStack

The foundation of `scienceStack` is the `pstack` plugin developed by Lauren Tan for Cursor. `pstack` pioneered autonomous subagent orchestration, dynamic playbook matching, and strict anti-slop guidelines.

In `scienceStack_antigravity`, this framework has been completely re-engineered for the Google Antigravity runtime and specialized for scientific and mathematical computing:

Component	**Original pstack (Cursor)**	**scienceStack (Antigravity)**
Primary Orchestrator	`poteto-mode`	`science-mode`
Worker Delegate	`poteto-agent`	`science-master`
Comment Reviewer	`comment-sicko`	`comment-master`
Theoretical Auditor	*None*	`theory-reviewer`
Subagent Dispatch	Synchronous / Cursor `Task`	Asynchronous `invoke_subagent`
Decision Logging	Local TSV / commit notes	Antigravity Brain Artifacts
Scientific Playbooks	Generic feature/bugfix	`equation-implementation`, `numerical-verification`
Benchmark Suite	Generic testing	`golden-dataset-benchmark`

Table 1.1: Evolution from Cursor `pstack` to Antigravity `scienceStack`.

Chapter 2

Installation, Setup, and Model Routing

2.1 Installing the Plugin

`scienceStack` conforms to the Agent Plugins 1.0.0 Specification. It can be installed globally into Antigravity or linked directly into project workspaces.

2.1.1 Global Installation

To install `scienceStack` globally across all Antigravity sessions on your machine, execute:

```
# Validate strict compliance first
agy plugin validate /home/seo3919/AI_skills/scienceStack_antigravity
   --strict

# Install into Antigravity CLI plugin engine
agy plugin install /home/seo3919/AI_skills/scienceStack_antigravity
```

2.1.2 Verification of Installed Components

Confirm that the plugin and its agents are active:

```
agy plugin list
# Expected output:
# - sciencestack (enabled, 0.15.29)
#   Components: 35 skills, 3 agents, 1 rule, 1 hook

agy agent
# Expected output:
# Available agents:
# comment-master
# science-master
# theory-reviewer
```

2.2 Model Configuration and Reasoning Budgets

2.2.1 The Role-Based Model Architecture

Scientific workflows require heterogeneous cognitive capabilities. Compiling numerical kernels, writing unit tests, and verifying syntax requires fast, low-latency execution. Conversely, proving

coordinate transformation invariance, analyzing ill-conditioned eigenspaces, and architecting modular solver pipelines requires maximum reasoning depth.

`scienceStack` defines clear roles mapped to Antigravity model tiers:

Role	Default Model	Antigravity Tier	Typical Tasks
`code`	`gemini-3.8-flash-high`	`Model: "flash"`	Kernels, MMS tests, refactoring, benchmark runners
`judgment`	`claude-opus-4-6-thinking`	`Model: "pro"`	Architecture, mathematical proofs, dimensional audits
`theory-reviewer`	`claude-opus-4-6-thinking`	`Model: "pro"`	Equation verification, sign checking, invariant bounds
`comment-master`	`gemini-3.8-flash-high`	`Model: "flash"`	Dead code removal, docstring formatting
`fast-worker`	`gemini-3.8-flash-medium`	`Model: "flash_lite"`	Linting, single-fixture test loops, syntax checks

Table 2.1: Model tier mapping in Antigravity.

2.2.2 Interactive Setup with /setup-sciencestack

Run the interactive setup skill to configure model preferences and reasoning budgets:

```
/setup-sciencestack
```

The skill queries the available models in your Antigravity environment, prompts for your target reasoning budget (`small`, `medium`, `large`, `unlimited`), and persists the configuration rule to:

`~/.gemini/config/rules/pstack-models.md`

Any role set to `auto` or `inherit-parent` dynamically inherits the parent chat session's active model.

Chapter 3

The Specialized Subagent Ecosystem

3.1 Overview of Agent Roles

Antigravity executes subagents asynchronously in background worker processes. `scienceStack` provides three purpose-built subagent personas that interact to implement, audit, and clean scientific software.

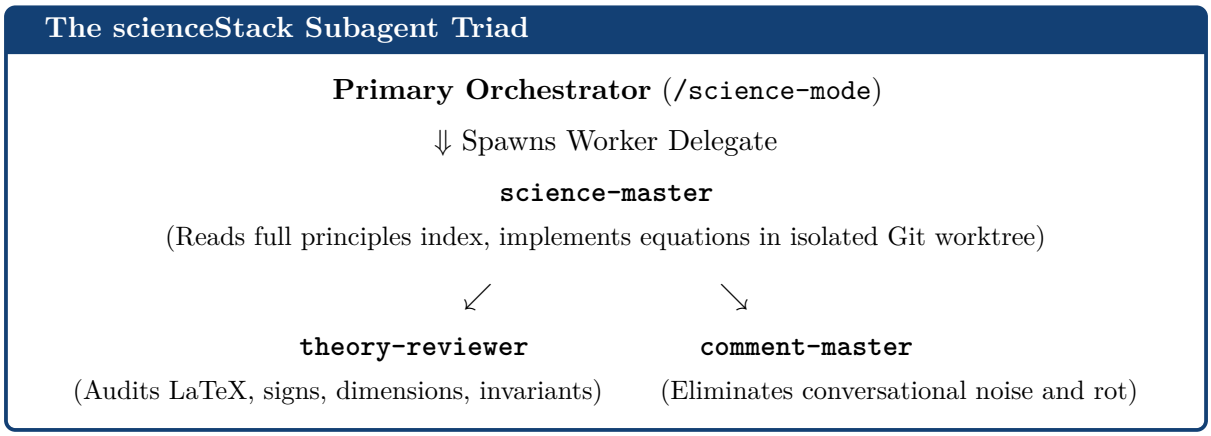


Figure 3.1: Interaction model among scienceStack subagents.

3.2 science-master: The Scientific Implementation Delegate

File: `agents/science-master.md`
Invocation: `invoke_subagent({ TypeName: "science-master", Role: "...", Prompt: "..." })`

`science-master` is the rigorous implementation engine. Whenever `science-mode` delegates a concrete coding assignment—such as implementing a finite difference stencil, constructing a sparse stiffness matrix, or writing an integration harness—it spawns `science-master`.

Antigravity Runtime Directive: science-master Operational Protocol

Before executing any file write or command, `science-master` is mandated to:

1. Read `skills/science-mode/SKILL.md` in full, including its inline Principles index.
2. Check out an isolated branch (`Workspace: "branch"`) to protect the main working directory.
3. Formulate continuous equations in LaTeX before implementing discrete algorithms.
4. Run test fixtures locally and return a consolidated diff summary to the parent orchestrator.

3.3 theory-reviewer: The Mathematical Auditor

File: `agents/theory-reviewer.md`
Invocation: `invoke_subagent({ TypeName: "theory-reviewer", Model: "pro", ... })`

`theory-reviewer` is a specialized, read-only auditor that acts as an adversarial theoretical reviewer. It never writes production code; its sole responsibility is auditing proposed diffs against governing physical laws and equations.

3.3.1 The 6-Point Theoretical Audit Checklist

When auditing a diff, `theory-reviewer` executes a rigorous verification pass:

1. **Continuous Equation Provenance**: Does every mathematical function specify its continuous formulation in LaTeX in docstrings, citing the textbook or paper equation number?
2. **Coordinate System & Sign Conventions**: Are spatial metrics (g_{ij}), coordinate systems (Cartesian, cylindrical, spherical), and sign conventions documented? For instance, in diffusion:
$$\frac{\partial u}{\partial t} = +\alpha \nabla^2 u \quad \text{(stable diffusion, } \alpha > 0\text{)} \tag{3.1}$$
A negative sign would represent reverse-time anti-diffusion, causing ill-posed exponential growth.
3. **Dimensional Homogeneity**: Do physical units match on both sides of every assignment? Are arguments to transcendental functions strictly dimensionless:
$$\exp\left(-\frac{E_a}{k_B T}\right) \quad \text{where} \quad \left[\frac{E_a}{k_B T}\right] = 1 \tag{3.2}$$
4. **Tensor and Index Order**: Are multi-dimensional arrays adhering to explicit indexing standards (row-major vs column-major, Einstein summation)?
5. **Normalization Constants**: Are integral normalizations preserved? E.g., for Fourier transforms:
$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i x \xi} dx \implies f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi) e^{2\pi i x \xi} d\xi \tag{3.3}$$
Or probability densities: $\int_\Omega |\psi(\mathbf{x}, t)|^2 d\mathbf{x} \equiv 1$.
6. **Physical Invariant Conservation**: Does the discrete formulation preserve Noetherian invariants (mass, energy, momentum, symplecticity) over simulation time?

3.4 comment-master: Eliminating Commentary Rot

File: `agents/comment-master.md`
Invocation: `invoke_subagent({ TypeName: "comment-master", Model: "flash", ... })`

Comments that narrate the obvious, defend sloppy design, or preserve dead code degrade repository quality. `comment-master` conducts automated line-by-line comment sweeps:

- **Deleted**: Narrative voice ("Here we loop over nodes"), apologies ("Hack for now, fix later"), dead commented-out code blocks, obvious type restatements.
- **Preserved**: LaTeX equation docstrings, literature citations, coordinate/sign definitions, non-obvious mathematical invariants, and hardware-specific synchronization guards.

Chapter 4

The Six Core Scientific Principles

4.1 Principle 1: Equation Provenance and Formal Documentation

Principle File: `skills/science-mode/principles/equation-provenance.md`

Every mathematical function or kernel in a scientific codebase must declare its provenance. Code without equation provenance is untestable against theory.

Scientific Principle: Equation Provenance

Every scientific implementation must:

1. State the continuous governing equation in LaTeX within the function docstring.
2. Cite the source paper or textbook with explicit chapter, page, or equation number.
3. Document coordinate system, metric tensor, and sign conventions.

4.1.1 Code Comparison: Bad vs. Compliant

```
# Bad: No provenance, ambiguous variables, undocumented formula
def calc_stress(e, nu, E):
    return E / ((1 + nu) * (1 - 2*nu)) * ((1 - nu) * e[0] + nu * e[1])
```

Listing 4.1: Non-compliant ungrounded function

```
def plane_strain_normal_stress(
    strain_xx: float,
    strain_yy: float,
    youngs_modulus: float,
    poissons_ratio: float
) -> float:
    r"""
    Compute normal stress \sigma_{xx} under isotropic linear elastic
       plane strain.

    Governing Continuous Equation:
    \begin{equation}
        \sigma_{xx} = \frac{E}{(1+\nu)(1-2\nu)} \left[
           (1-\nu)\varepsilon_{xx} + \nu\varepsilon_{yy} \right]
    \end{equation}

```

```
    Reference:
        Timoshenko, S. P., & Goodier, J. N. (1970). "Theory of
            Elasticity",
        3rd ed., McGraw-Hill, Chapter 2, Eq. (11.3), p. 34.

    Assumptions & Conventions:
        - Coordinate System: Cartesian (x, y, z)
        - Condition: Plane strain (\varepsilon_{zz} = \gamma_{xz} =
            \gamma_{yz} = 0)
        - Sign Convention: Tensile stress is positive (> 0).
    """
    E, nu = youngs_modulus, poissons_ratio
    coeff = E / ((1.0 + nu) * (1.0 - 2.0 * nu))
    return coeff * ((1.0 - nu) * strain_xx + nu * strain_yy)
```

Listing 4.2: Compliant function with Equation Provenance

4.2 Principle 2: Dimensional Discipline & Scaling

Principle File: `skills/science-mode/principles/dimensional-discipline.md`

Physical quantities possess dimensions in fundamental SI units ($[M], [L], [T], [\Theta], [I]$). Dimensional mismatch is proof of a mathematical bug.

Scientific Principle: Dimensional Discipline

1. Verify dimensional homogeneity: [LHS] = [RHS] for every physical relation.
2. Nondimensionalize equations using characteristic scales prior to numerical integration:
$$x^* = \frac{x}{L_0}, \quad t^* = \frac{tU_0}{L_0}, \quad p^* = \frac{p}{\rho_0 U_0^2} \tag{4.1}$$
3. Verify that arguments to transcendental functions ($\exp, \ln, \sin, \cos$) are strictly dimensionless ($[f(\text{arg})] = 1 \implies [\text{arg}] = 1$).

4.3 Principle 3: Preservation of Physical Conservation Laws

Principle File: `skills/science-mode/principles/preserve-conservation-laws.md`

According to Noether's theorem, every continuous physical symmetry corresponds to a conserved invariant:

- Time translation invariance $\implies$ Energy conservation ($\frac{dE}{dt} = 0$).
- Spatial translation invariance $\implies$ Linear momentum conservation ($\frac{d\mathbf{p}}{dt} = \mathbf{0}$).
- Spatial rotational invariance $\implies$ Angular momentum conservation ($\frac{d\mathbf{L}}{dt} = \mathbf{0}$).
- Phase invariance (gauge symmetry) $\implies$ Charge / probability norm conservation ($\int |\psi|^2 dx = 1$).

Standard dissipative discrete schemes (such as implicit backward Euler) artificially dissipate energy, while explicit schemes often artificially inject energy. `scienceStack` mandates **invariant tracking**:

$$\delta(t) = \frac{|I(t) - I(0)|}{|I(0)|} \leq \tau_{\text{drift}} \tag{4.2}$$

4.4 Principle 4: Numerical Stability First

Principle File: `skills/science-mode/principles/numerical-stability-first.md`

Numerical algorithms must remain stable in floating-point arithmetic (IEEE-754). Premature vectorization must never compromise numerical conditioning.

Numerical Safety Alert: Catastrophic Cancellation

Subtracting two nearly equal floating-point numbers $a \approx b$ causes severe cancellation, wiping out significant bits:

$$\text{Error}_{\text{rel}} \approx \frac{\epsilon_{\text{mach}}}{|a - b|} \tag{4.3}$$

Use conditioned mathematical identities or special functions:

- Replace $1 - \cos x$ with $2\sin^2(x/2)$ for $|x| \ll 1$.
- Replace $\ln(1 + x)$ with `math.log1p(x)`.
- Replace $e^x - 1$ with `math.expm1(x)`.
- Replace $\ln \sum e^{x_i}$ with `scipy.special.logsumexp(x)`.

Numerical Safety Alert: Strict Matrix Inversion Prohibition

Never compute $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ via `numpy.linalg.inv(A) @ b`. Matrix inversion introduces an operation count of $2n^3$ flops and squares the numerical condition number:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\| \tag{4.4}$$

Always compute linear solves via direct factorization or Krylov iterative methods:

```
# FORBIDDEN:
x = np.linalg.inv(A) @ b

# MANDATED:
x = np.linalg.solve(A, b)
# Or for symmetric positive definite matrices:
# L = scipy.linalg.cho_factor(A); x = scipy.linalg.cho_solve(L, b)
```

4.5 Principle 5: Verification Against Analytical Limits and Golden Benchmarks

Principle File: `skills/science-mode/principles/verify-analytical-limits.md`

Unit tests asserting that code "runs without throwing an exception" are insufficient for scientific software. Verification requires rigorous mathematical convergence:

1. **Analytical Closed-Form Solutions**: Validate against known exact solutions under simplified boundary conditions.
2. **Method of Manufactured Solutions (MMS)**: For nonlinear or coupled PDEs without closed-form solutions, manufacture an analytical solution $u_{\text{exact}}(\mathbf{x}, t)$, substitute into the PDE to derive an analytical source term f_{mms}, and verify that the discrete solver converges

to u_{exact} at the theoretical asymptotic rate $O(h^p)$:

$$p = \frac{\ln(E_{h_1}/E_{h_2})}{\ln(h_1/h_2)} \tag{4.5}$$

3. **Golden Datasets**: Validate against community-accepted experimental or high-resolution numerical datasets stored in `tests/golden/`.

4.6 Principle 6: Physics-Modular Architecture & Decoupled Operators

Principle File: `skills/science-mode/principles/physics-modular-architecture.md`

Scientific simulation software frequently degrades into monolithic codebases where physics governing laws (e.g., gravitational accelerations, nuclear reaction networks, equations of state, radiative transfer opacity, stress-strain tensors) are tightly entangled with numerical time steppers (Runge-Kutta, Leapfrog), spatial mesh topologies, and file I/O operations.

When physical models are monolithic, external research codes, alternate integrators, and higher-order solvers cannot invoke the physics routines independently. `scienceStack` mandates a strict **Physics-Modular Architecture**:

Scientific Principle: Physics-Modular Architecture

1. **Decouple Physics from Numerics**: Isolate pure physical laws into dedicated, stateless modules (`physics/`). Physics operators must accept pure state vectors $\mathbf{y}$ and physical parameters $\mathbf{p}$, computing state derivatives $\frac{d\mathbf{y}}{dt}$, accelerations $\mathbf{a}_i$, or fluxes $\mathbf{F}(\mathbf{u})$ without time-stepping logic or grid allocations:

$$\mathbf{a}_i = \mathcal{F}_{\text{phys}}(\{\mathbf{q}_j\}, \{m_j\}, G, \epsilon) \tag{4.6}$$

2. **Generalized Numerical Steppers**: Numerical integrators (e.g., symplectic Leapfrog, Radau, BDF) must be physics-agnostic, accepting an arbitrary callable physics operator:

$$\mathbf{y}^{n+1} = \mathcal{I}_{\text{stepper}}(\mathbf{y}^n, \Delta t, \mathcal{F}_{\text{phys}}) \tag{4.7}$$

3. **Driver-Agnostic Composability**: Ensure external solvers or other analysis codes can directly import and execute the physics module without instantiating simulation drivers.

4.6.1 Architectural Comparison: Monolithic vs. Modular

```
# BAD: Gravitational force calculation is inextricably tangled inside
# the simulation loop and file output logic. Other codes cannot reuse
    this physics.
class MonolithicSimulation:
    def step_and_save(self, dt, filepath):
        for i in range(self.n):
            for j in range(self.n):
                # Physics formula embedded inside stepper...
                ...
        self.positions += self.velocities * dt
        self.save_to_disk(filepath)
```

Listing 4.3: Non-compliant monolithic coupling

```python
# GOOD:
# 1. Pure physics module: callable by ANY solver or analysis tool
# physics/gravity.py
def gravitational_accelerations(
    positions: List[List[float]],
    masses: List[float],
    G: float = 1.0,
    softening: float = 0.0
) -> List[List[float]]:
    r"""Stateless physics operator: a_i = -G \sum_j m_j (q_i - q_j) / |q_i - q_j|^3."""
    ...
    return accelerations

# 2. Generalized numerical stepper (physics-agnostic)
# numerics/symplectic.py
def leapfrog_step(q, v, dt, accel_operator):
    r"""Symplectic Drift-Kick-Drift step accepting any acceleration operator."""
    q_half = q + 0.5 * dt * v
    v_next = v + dt * accel_operator(q_half)
    q_next = q_half + 0.5 * dt * v_next
    return q_next, v_next
```

Listing 4.4: Compliant physics-modular decomposition

Chapter 5

Scientific Skills and Playbooks

5.1 The Master Controller: /science-mode

`/science-mode` is the entry point for all non-trivial scientific tasks. When invoked with a user objective, it:

1. Analyzes the prompt and classifies the intent into a specialized playbook.
2. Initializes an in-context todo list or Brain Artifact.
3. Evaluates prerequisites (models, verification harnesses, git branch state).
4. Spawns subagents (`science-master`, `theory-reviewer`, `comment-master`) as each step requires.
5. Collects empirical evidence before declaring completion.

5.2 Dedicated Scientific Skills

5.2.1 /scientific-equation-verifier

Directory: `skills/scientific-equation-verifier/SKILL.md`

Performs an exhaustive audit of an existing codebase or proposed diff against mathematical and physical principles.

Usage:

```
/scientific-equation-verifier verify the stress-strain constitutive
    updates in elasticity.py
```

5.2.2 /numerical-stability-audit

Directory: `skills/numerical-stability-audit/SKILL.md`

Audits numerical algorithms for floating-point vulnerabilities:

- Matrix condition number estimates ($\kappa_2(\mathbf{A})$).
- Detects explicit matrix inversions (`np.linalg.inv`).
- Identifies unguarded divisions by zero in geometric and kinetic factors.
- Analyzes stiff ordinary differential equations (ODEs) to ensure appropriate implicit integrators (e.g., Radau IIA, BDF) are selected over explicit Runge-Kutta.

5.2.3 /golden-dataset-benchmark

Directory: `skills/golden-dataset-benchmark/SKILL.md`

Manages the acquisition, generation, and verification of canonical test problems across major scientific disciplines.

Domain	Canonical Benchmark	Validation Metric
Astronomy / Orbits	2-Body Kepler Problem	Vis-viva energy error $< 10^{-14}$; specific angular momentum error $< 10^{-14}$
Astronomy / N-body	Symplectic Planetary Orbit	Zero secular energy drift over 10^4 steps ($\lvert\Delta E/E_0\rvert < 10^{-10}$); angular momentum conserved
Astronomy / Stars	Lane-Emden Polytropes	Surface radius ξ_1 and mass factor ω_n matching exact values for $n = 0, 1$ to $< 10^{-10}$ and tabulated benchmarks ($n = 1.5, 3.0$) to $< 10^{-5}$
Astronomy / Frames	IAU Galactic to ICRS	Orthonormality error $\lVert\mathbf{R}^T\mathbf{R} - \mathbf{I}\rVert < 10^{-14}$, $\det \mathbf{R} = 1.0$
CFD	1D Sod Shock Tube	L_1 and L_2 norm vs. exact Riemann solver; shock position $\Delta x < 0.5\Delta x_{\text{cell}}$
CFD / Turbulence	2D Taylor-Green Vortex	Kinetic energy decay rate $dK/dt = -2\nu\Omega$
Solid Mechanics (FEM)	Cook's Membrane	Vertical tip deflection u_y at $(48, 60)$ under mesh refinement
Chemical Kinetics	Robertson Reaction System	Mass conservation $\lvert y_1 + y_2 + y_3 - 1.0\rvert < 10^{-12}$; stiffness ratio $\approx 10^9$
Molecular Dynamics	Lennard-Jones Cluster	Microcanonical energy drift $\lvert\Delta E(t)/E_0\rvert < 10^{-5}$ over 10^5 steps
Quantum Mechanics	1D Harmonic Oscillator	Energy eigenvalue error $\lvert E_n - \hbar\omega(n + 1/2)\rvert < 10^{-8}$

Table 5.1: Canonical benchmark problems supported by /golden-dataset-benchmark.

5.3 Specialized Scientific Playbooks

Inside `skills/science-mode/playbooks/`, `scienceStack` ships specialized playbooks that structure the lifecycle of scientific implementations:

5.3.1 Playbook: equation-implementation

Path: `skills/science-mode/playbooks/equation-implementation.md`

1. **Theory Extraction**: Isolate the governing continuous differential/integral equations, boundary conditions, and sign conventions from literature.
2. **Coordinate & Tensor Settlement**: Settle metric components, array layouts, and dimensionless scaling factors.
3. **Analytical Limit Definition**: Formulate exact asymptotic solutions or MMS test fixtures.
4. **Numerical Implementation**: Spawn `science-master` to write clean, conditioned code without explicit inversions.
5. **Theoretical Audit**: Spawn `theory-reviewer` to verify signs, normalizations, and LaTeX provenance.
6. **Convergence Testing**: Prove grid convergence $O(h^p)$ against the analytical limit.

5.3.2 Playbook: numerical-verification

Path: `skills/science-mode/playbooks/numerical-verification.md`

1. **Refinement Hierarchy**: Construct grids with cell sizes $h, h/2, h/4, h/8$.
2. **Discrete Error Computation**: Measure L_2 error norm $E(h) = \|u_h - u_{\text{exact}}\|_{L_2}$.
3. **Empirical Order Calculation**: Compute convergence slope $p = \frac{\ln(E_{2h}/E_h)}{\ln 2}$.
4. **Conservation Monitoring**: Record invariant drift $\Delta I(t)$ across simulation duration.
5. **Artifact Generation**: Output convergence tables and error plots to Brain Artifacts.

Chapter 6

Astronomy and Celestial Mechanics Solvers

6.1 Overview of the Astronomy Suite

Astronomy, astrophysics, and celestial mechanics impose stringent mathematical demands on computational algorithms. Planetary and stellar systems are Hamiltonian and non-dissipative; standard explicit integrators introduce artificial secular energy drift that destroys orbital stability over astronomical integration intervals. Coordinate transformations require direction cosine matrices in SO(3) that must preserve unit norm and orthogonality to machine precision. Stellar structure equations exhibit coordinate singularities at the stellar core ($\xi = 0$) that require analytical series expansions before numerical stepping.

`scienceStack` provides four bundled, zero-network reference solvers in `benchmarks/astronomy/`:

1. **`kepler_solver.py`**: Universal Kepler equation solver for elliptic and hyperbolic motion with cubic Halley iteration and Cartesian orbit propagation.
2. **`symplectic_nbody.py`**: Symplectic Leapfrog and Yoshida integrators for gravitationally interacting N-body systems with exact energy and angular momentum tracking.
3. **`lane_emden_solver.py`**: Stellar structure polytropic solver with near-center series expansion removing the $2/\xi$ coordinate singularity.
4. **`coordinate_transforms.py`**: Direction cosine transformations between Equatorial (J2000 ICRS), Ecliptic, and Galactic coordinate systems.

6.2 Kepler Orbit Propagation and Vis-Viva Invariants

6.2.1 Mathematical Formulation

For two-body gravitational motion under central potential $V(r) = -\mu/r$, Kepler's equation for elliptic orbits ($0 \leq e < 1$) relates mean anomaly $M(t) = n(t - t_0)$ to eccentric anomaly E:

$$M = E - e \sin E \tag{6.1}$$

The true anomaly ν is obtained without quadrant ambiguity or cancellation via:

$$\tan \frac{\nu}{2} = \sqrt{\frac{1+e}{1-e}} \tan \frac{E}{2} \tag{6.2}$$

The orbital radius and perifocal state vector are:

$$r = a(1 - e\cos E), \quad \mathbf{r}_{\text{orb}} = \begin{bmatrix} r\cos\nu \\ r\sin\nu \\ 0 \end{bmatrix}, \quad \mathbf{v}_{\text{orb}} = \frac{\mu}{h}\begin{bmatrix} -\sin\nu \\ e + \cos\nu \\ 0 \end{bmatrix} \tag{6.3}$$

where $h = \sqrt{\mu a(1-e^2)}$ is the specific angular momentum.

6.2.2 Cubic Halley Solver

For high eccentricities ($e \to 1$), standard Newton-Raphson oscillates or converges slowly. `scienceStack` implements third-order Halley acceleration:

$$\Delta_1 = -\frac{f(E)}{f'(E)}, \quad \Delta_{\text{Halley}} = -\frac{f(E)}{f'(E) + \frac{1}{2}\Delta_1 f''(E)} \tag{6.4}$$

where $f(E) = E - e\sin E - M$, $f'(E) = 1 - e\cos E$, and $f''(E) = e\sin E$. This method achieves machine precision ($|E_{k+1} - E_k| < 10^{-14}$) in 3 iterations.

6.2.3 Invariant Verification

Every propagated state $(\mathbf{r}, \mathbf{v})$ must satisfy the vis-viva equation:

$$\mathcal{E} = \frac{v^2}{2} - \frac{\mu}{r} = -\frac{\mu}{2a} \tag{6.5}$$

and specific angular momentum conservation $\mathbf{h} = \mathbf{r} \times \mathbf{v}$. The reference solver asserts that both invariants deviate by less than 10^{-14} in double precision.

6.3 Symplectic Gravitational N-Body Dynamics

6.3.1 The Hamiltonian and Symplectic Structure

An N-body gravitational system is governed by the Hamiltonian:

$$H(\mathbf{q}, \mathbf{p}) = \sum_{i=1}^{N} \frac{\mathbf{p}_i^2}{2m_i} - \sum_{1 \le i < j \le N} \frac{G m_i m_j}{\|\mathbf{q}_i - \mathbf{q}_j\|} \tag{6.6}$$

Standard integrators (such as Runge-Kutta 4th order) are non-symplectic: they fail to preserve the differential 2-form $\omega = \sum_i d\mathbf{q}_i \wedge d\mathbf{p}_i$. Over thousands of orbital periods, non-symplectic integrators introduce artificial dissipation or excitation, causing orbits to decay or blow up.

6.3.2 Drift-Kick-Drift Leapfrog

`scienceStack` implements the symplectic 2nd-order Drift-Kick-Drift (DKD) scheme:

$$\mathbf{q}_i^{n+1/2} = \mathbf{q}_i^n + \frac{\Delta t}{2}\mathbf{v}_i^n \tag{6.7}$$

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \Delta t\, \mathbf{a}_i(\mathbf{q}^{n+1/2}) \tag{6.8}$$

$$\mathbf{q}_i^{n+1} = \mathbf{q}_i^{n+1/2} + \frac{\Delta t}{2}\mathbf{v}_i^{n+1} \tag{6.9}$$

where gravitational accelerations $\mathbf{a}_i = -G\sum_{j\neq i} m_j \frac{\mathbf{q}_i - \mathbf{q}_j}{\|\mathbf{q}_i - \mathbf{q}_j\|^3}$ satisfy Newton's third law ($\mathbf{F}_{ij} = -\mathbf{F}_{ji}$). Because DKD is symplectic, the numerical trajectory exactly traces a nearby "shadow Hamiltonian" $\tilde{H} = H + O(\Delta t^2)$, ensuring that total energy $E(t)$ oscillates boundedly with zero secular growth over 10^5 steps.

6.4 Stellar Structure: Lane-Emden Polytropes

6.4.1 Governing Equation and Singularity Removal

Hydrostatic equilibrium and a polytropic equation of state ($P = K\rho^{1+1/n}$) yield the dimensionless Lane-Emden equation:

$$\frac{1}{\xi^2}\frac{d}{d\xi}\left(\xi^2\frac{d\theta}{d\xi}\right) + \theta^n = 0 \iff \frac{d^2\theta}{d\xi^2} + \frac{2}{\xi}\frac{d\theta}{d\xi} + \theta^n = 0 \tag{6.10}$$

with central boundary conditions $\theta(0) = 1$ and $\theta'(0) = 0$. At $\xi = 0$, the geometry term $(2/\xi)\theta'$ creates a coordinate singularity. `scienceStack` removes this singularity by starting integration at $\xi_0 = 10^{-4}$ using the analytic series expansion:

$$\theta(\xi) \approx 1 - \frac{\xi^2}{6} + \frac{n\xi^4}{120} - \frac{n(8n-5)\xi^6}{15120} + O(\xi^8) \tag{6.11}$$

6.4.2 Analytical and Tabulated Benchmarks

The numerical solver integrates radially outward until $\theta(\xi_1) = 0$, determining the stellar radius ξ_1 and mass factor $\omega_n = -\xi_1^2\theta'(\xi_1)$:

- $n = 0$ (Constant density): $\xi_1 = \sqrt{6} \approx 2.449490$, $\omega_0 = 2\sqrt{6} \approx 4.898979$.
- $n = 1$ (Linear polytrope): $\xi_1 = \pi \approx 3.141593$, $\omega_1 = \pi \approx 3.141593$.
- $n = 1.5$ (Convective stars / Non-relativistic white dwarfs): $\xi_1 \approx 3.65375$, $\omega_{1.5} \approx 2.71406$.
- $n = 3.0$ (Radiation-dominated stars / Relativistic Chandrasekhar limit): $\xi_1 \approx 6.89685$, $\omega_3 \approx 2.01824$.

Chapter 7

Automated Tooling and Antigravity Lifecycle Hooks

7.1 The AST Numerical Stability Auditor

Script: `scripts/audit_numerical_ast.py`

To ensure deterministic compliance before code is committed or tested, `scienceStack` includes an Abstract Syntax Tree (AST) static analysis tool. The auditor parses Python source code and identifies:

1. **Explicit Matrix Inversions** (`BAN_MATRIX_INV`): Flags occurrences of `np.linalg.inv` or `scipy.linalg.inv`. Suggests direct factorized solvers (`solve(A, b)`).
2. **Catastrophic Cancellation Hazards** (`CANCELLATION_LOG1P`, `CANCELLATION_EXPM1`, `CANCELLATION_ONE_MINUS_COS`): Detects expressions like $\log(1 + x)$ and $e^x - 1$ where $|x| \ll 1$ causes loss of significant bits.
3. **Single-Precision Traps** (`PRECISION_FLOAT32`): Detects single-precision casting in time-steppers or simulation loops where double precision is required.
4. **Missing LaTeX Docstrings** (`MISSING_LATEX_PROVENANCE`): Flags scientific kernels lacking continuous equations in LaTeX docstrings.

7.2 Symbolic Method of Manufactured Solutions (MMS) Generator

Script: `scripts/mms_generator.py`

Verifying PDE solvers against analytical limits requires exact forcing terms. The MMS generator uses `SymPy` to symbolically substitute any manufactured exact solution $u_{\text{exact}}(\mathbf{x}, t)$ into a differential operator $\mathcal{L}[u]$:

```
python3 scripts/mms_generator.py \
  --exact "sin(pi*x) * exp(-t)" \
  --operator "diff(u, t) - alpha * diff(u, x, 2)" \
  --params "alpha=0.1" \
  --output test_mms_heat.py
```

The tool outputs the simplified analytical forcing term $f(x, t) = \mathcal{L}[u_{\text{exact}}]$, formats the continuous equation in LaTeX, and writes a complete, executable `pytest` test harness verifying the asymptotic convergence order $O(h^p)$.

7.3 Symbolic Jacobian Synthesizer

Script: `scripts/derive_jacobian.py`

In stiff kinetic systems or implicit finite element formulations, hand-deriving Jacobians is a major source of sign errors and transposed indices. The Jacobian synthesizer computes analytical partial derivatives $\mathbf{J}_{ij} = \partial f_i / \partial y_j$, simplifies expressions symbolically, and emits vectorized, conditioned Python/NumPy code directly.

7.4 Native Antigravity Lifecycle Hook: PreToolUse Safety Gate

File: `hooks.json` & `scripts/hook_pre_tool_guard.py`

Antigravity executes lifecycle hooks at defined stages of agent execution. `scienceStack` registers a native `PreToolUse` hook for `write_file` and `edit_file` tools. When an agent attempts to write or edit a Python or scientific code file:

1. The hook parses the incoming JSON tool payload from `stdin`.
2. It inspects the proposed content for forbidden patterns, specifically explicit matrix inversions (`np.linalg.inv`).
3. If a violation is detected, the hook halts the tool step immediately:

```
{
  "decision": "deny",
  "reason": "[scienceStack Safety Gate] Explicit matrix
     inversion ('inv') detected..."
}
```

4. If compliant, the hook permits execution (`"decision": "allow"`).

This ensures that dangerous numerical patterns are blocked deterministically before being written to disk.

Chapter 8

End-to-End Practical Tutorials

8.1 Tutorial 1: Implementing a Stiff Kinetic System (Robertson)

8.1.1 Problem Statement

The Robertson chemical reaction model represents an autocatalytic reaction with extreme timescale separation (stiffness ratio $\approx 10^9$). The governing system is:

$$\frac{dy_1}{dt} = -0.04y_1 + 10^4 y_2 y_3 \tag{8.1}$$

$$\frac{dy_2}{dt} = 0.04y_1 - 10^4 y_2 y_3 - 3 \cdot 10^7 y_2^2 \tag{8.2}$$

$$\frac{dy_3}{dt} = 3 \cdot 10^7 y_2^2 \tag{8.3}$$

with initial conditions $\mathbf{y}(0) = [1, 0, 0]^T$ over $t \in [0, 10^5]$. Total mass is strictly conserved:

$$y_1(t) + y_2(t) + y_3(t) \equiv 1.0 \quad \forall t \geq 0 \tag{8.4}$$

8.1.2 Executing with /science-mode

To instruct Antigravity to implement and verify this solver, type:

```
/science-mode implement the Robertson stiff chemical kinetics
   benchmark in kinetics.py.
Use an implicit integrator with Jacobian evaluation.
Verify that total mass y1 + y2 + y3 is conserved to within 1e-12.
Validate against reference data in tests/golden/robertson.
```

8.1.3 What Happens Internally

1. `science-mode` classifies the prompt into the `equation-implementation` playbook.
2. It verifies that `tests/golden/robertson/` exists or downloads reference trajectories.
3. It invokes `science-master` in an isolated branch. `science-master` implements the RHS and analytic Jacobian $\mathbf{J}(t, \mathbf{y})$:

$$\mathbf{J} = \begin{bmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \cdot 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \cdot 10^7 y_2 & 0 \end{bmatrix} \tag{8.5}$$

4. `theory-reviewer` audits the Jacobian line-by-line against the partial derivatives $\partial \dot{y}_i / \partial y_j$.

5. `numerical-stability-audit` verifies that Radau IIA / BDF integration is selected and explicit RK45 is rejected.

6. Test fixtures verify that the mass conservation invariant drift remains below 10^{-12}.

8.2 Tutorial 2: Conducting an Overnight Autonomous Run

When conducting an extensive simulation study, refactoring an entire solver module, or tuning convergence across a multi-mesh grid, you can initiate an autonomous overnight run.

8.2.1 The Overnight Contract Prompt

```
/science-mode i'm stepping away. refactor the finite volume flux
   splitters in cfd/fluxes.py
to use Roe and HLLC approximate Riemann solvers.
Do not stop until:
1. All golden tests in tests/golden/sod_shock/ pass with L2 relative
   error < 0.02.
2. Invariant conservation of total mass and total energy is verified
   over 500 time steps.
3. theory-reviewer signs off on sign conventions and normalizations.
Keep a decision log in brain/artifacts so I can audit everything in
   the morning.
```

8.2.2 Safeguards During Autonomous Execution

- **Non-blocking Reactive Execution**: Antigravity does not freeze or block your shell; background tasks trigger reactive resume events.

- **Decision Trail**: The `show-me-your-work` mechanism logs every design decision, alternative rejected, and benchmark metric to a structured Antigravity Brain Artifact.

- **Self-Correction**: If a test fails with numerical instability, the agent triggers the stability audit skill (`/numerical-stability-audit`), pinpoints the flux limiter or time-step CFL violation, adjusts the formulation, and re-executes tests autonomously.

8.3 Tutorial 3: Verifying Symplectic N-Body Orbit Invariants

8.3.1 Problem Statement

In simulating planetary orbits (e.g. Earth around Sun), explicit non-symplectic integrators introduce secular energy growth that causes the planet to artificially spiral outward. The objective is to verify an astronomical N-body simulator using the symplectic Leapfrog scheme and prove zero secular energy growth over 10^4 integration steps.

8.3.2 Executing with /science-mode

```
/science-mode implement a symplectic 2-body orbit propagator in
   orbit_sim.py.
Use the Drift-Kick-Drift Leapfrog integrator.
Verify that:
1. Relative energy drift |E(t) - E(0)| / |E(0)| stays below 1e-10 over
   10000 steps.
```

```
2. Angular momentum is conserved to within 1e-14.
3. Validate orbital elements conversion against
   benchmarks/astronomy/kepler_solver.py.
```

8.3.3 Verification and Invariant Output

1. `science-mode` routes the task to the `equation-implementation` playbook and loads the `astronomy-computing` skill.
2. `science-master` implements the Drift-Kick-Drift scheme and energy tracking in `orbit_sim.py`.
3. `theory-reviewer` audits the gravitational acceleration formulas to verify Newton's third law ($\mathbf{F}_{ji} = -\mathbf{F}_{ij}$) to prevent artificial momentum creation.
4. Tests demonstrate that the energy drift oscillates boundedly at $\sim 5 \times 10^{-11}$ without any secular slope, confirming symplectic structure preservation.

Chapter 9

Antigravity Technical Adapter Reference

9.1 Runtime Tool Mapping

The `rules/AGENTS.md` rule automatically injected by `scienceStack` instructs Antigravity how to translate Agent Plugin conventions into native Antigravity primitives:

Cursor / Standard Tool	Antigravity Equivalent	Operational Directives
`Task`	`invoke_subagent`	Background execution; sets `TypeName` and `Role`.
`AskQuestion`	`ask_question`	Multi-choice interactive prompts.
`TodoWrite`	Brain Artifacts	Persistent Markdown state in `<appDataDir>/brain/<id>/`.
`Bash`	`run_command`	Shell execution (`rg`, `find`, `pytest`).
`Read / Edit`	`read_file / edit_file`	Exact character-sequence replacement tools.
`environment: "cloud"`	`Workspace: "branch"`	Clones/branches workspace for clean isolation.
`environment: "local"`	`Workspace: "inherit"`	Shares root repository directly.

Table 9.1: Adapter tool mapping in Google Antigravity.

9.2 Antigravity Brain Artifacts

All audit trails, convergence tables, and execution summaries generated by `scienceStack` are written to the Antigravity Artifact directory:

`<appDataDir>/brain/<conversation-id>/`

These artifacts support GitHub-style alerts (`NOTE`, `TIP`, `IMPORTANT`, `WARNING`, `CAUTION`), interactive Mermaid flowcharts, and embedded convergence plots.

Chapter 10

Complete Skill and Playbook Catalog

10.1 Primary Scientific and Orchestration Skills

Skill / Slash Command	Description & Purpose
`/science-mode`	Primary orchestrator. Matches user prompt to playbooks, manages subagents, and drives tasks to empirical verification.
`/setup-sciencestack`	Configures model choices and reasoning budgets per role; generates `pstack-models.md`.
`/science-help`	Interactive guide. Recommends skills and drafts prompts for any coding situation.
`/scientific-equation-verifier`	6-point theoretical audit for LaTeX provenance, signs, dimensions, and normalizations.
`/numerical-stability-audit`	Deep inspection for condition numbers, catastrophic cancellation, zero division, and stiff ODEs.
`/golden-dataset-benchmark`	Catalog of canonical test problems (CFD, FEM, MD, quantum) and reference dataset acquisition.
`/astronomy-computing`	Specialized algorithms and invariants for astronomy, orbital mechanics (Kepler, symplectic N-body), stellar structure (Lane-Emden), and celestial SO(3) coordinate frames.
`/precision-audit`	Audits floating-point precision hazards (float32 vs float64), condition numbers, and mixed-precision iterative refinement.
`/accelerated-scientific-computing`	Memory layouts (row vs column major), SIMD vectorization, and precision guards for C++, Fortran, Julia, and JAX.

10.2 Scientific and Engineering Principles Catalog

Architectural Note: Token-Optimized Principle Architecture

To eliminate discovery prompt bloat, the 30 principles are bundled under `skills/science-mode/principles/` rather than registered as individual top-level discovery skills. This saves over 1,300 prompt tokens per user message while remaining fully indexed and dynamically accessible by `science-mode` and subagents.

Principle	Rule & Core Mandate
`equation-provenance`	Continuous equation in LaTeX, literature citation, coordinate/sign documentation.
`dimensional-discipline`	Validate $[LHS] = [RHS]$, non-dimensionalization, dimensionless transcendental inputs.
`preserve-conservation-laws`	Explicitly monitor and conserve energy, mass, momentum, and norm invariants.
`numerical-stability-first`	Eliminate cancellation; ban `inv(A) @ b`; use conditioned linear solvers.
`verify-analytical-limits`	Method of Manufactured Solutions (MMS), closed-form benchmarks, $O(h^p)$ convergence.
`physics-modular-architecture`	Isolate pure physical laws, forces, and equations of state into callable modules.
`laziness-protocol`	Bias toward deletion and the smallest change that solves the problem.
`foundational-thinking`	Get data structures and core types right before writing algorithmic logic.
`redesign-from-first-principles`	Redesign as if the requirement were foundational from day one.
`attack-the-premise`	When multiple fixes fail, write down and test the shared underlying premise.
`subtract-before-you-add`	Remove dead code and redundant wrappers before building new features.
`minimize-reader-load`	Collapse one-caller wrappers and shrink mutable state scope.
`build-the-lever`	Build reproducible scripts/tools instead of performing work by hand.
`model-the-domain`	Encode the domain in a structure instead of scattered conditionals.
`boundary-discipline`	Validate at system boundaries; trust internal types and keep pure functions.
`type-system-discipline`	Make illegal states unrepresentable; brand semantic primitives; derive types.
`make-operations-idempotent`	Converge to the same end state regardless of partial prior runs.
`migrate-callers-then-delete`	Migrate callers and delete old APIs in the same wave.
`separate-before-serializing`	Eliminate state sharing first; serialize structurally only when needed.
`prove-it-works`	Verify against the real artifact and execution proof, never self-reports.
`fix-root-causes`	Trace symptoms back to the initial origin and fix them at the root.
`sequence-verifiable-units`	Break work into small units ending in verifiable states.
`test-behavior-not-impl`	Assert observable behavior, not internal implementation details.
`explain-the-number`	Name what limits a measured performance number and rule out artifacts.
`guard-the-context-window`	Route bulk reading to subagents; keep summaries in the main thread.
`never-block-on-the-human`	Proceed on reversible tasks; reserve prompts for irreversible actions.
`encode-lessons-in-structure`	Turn repeated advice into automated lints, hooks, or checks.

10.3 General Engineering and Review Skills

Skill / Slash Command	Description & Purpose
`/architect`	Settle caller usage, interfaces, types, and module shape before implementing code.
`/arena`	Run N parallel subagent attempts at the same design problem, then synthesize the best components.
`/swarm`	Deploy parallel subagents across slices, packages, or test suites, returning a single aggregated report.
`/interrogate`	Multi-model adversarial review of proposed diffs against safety, performance, and style standards.
`/tdd`	Test-driven bug reproduction: write a failing fixture first, verify failure, then implement fix.
`/no-comments`	Strip narrative comments before review; spawns `comment-master`.
`/unslop`	Clean conversational filler, apologies, and AI markers from documentation and code.
`/show-me-your-work`	Record reviewable decision trails to TSV files and Antigravity Brain Artifacts.
`/how`	Deep walkthrough of how an existing system or subsystem operates.
`/why`	Investigates architectural rationale using git history, issue records, and chat context.
`/teach`	Synthesizes `how` and `why` into a structured diagrammatic explanation.
`/ketchup`	Catch-up summary of chat changes since your last message with actionable decisions.
`/blast-radius`	Analyze downstream breakage risk outside the diff, proving safety with running tests.
`/benchmark-checklist`	Vets performance measurements (timers, warmups, repeat runs) before reporting numbers.

Recommended Literature & References

1. **Numerical Methods & Linear Algebra**:
   - Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
   - Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.
   - Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
2. **Partial Differential Equations & Verification**:
   - Roache, P. J. (2009). *Fundamentals of Verification and Validation*. Hermosa Publishers.
   - LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press.
   - Salari, K., & Knupp, P. (2000). *Code Verification by the Method of Manufactured Solutions*. Sandia National Laboratories Report SAND2000-1444.
3. **Ordinary Differential Equations & Chemical Kinetics**:
   - Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd ed. Springer.
   - Robertson, H. H. (1966). "The numerical integration of some systems of stiff ordinary differential equations." In *Numerical Analysis: An Introduction*, pp. 178–182.